1 - Modelagem de banco de dados relacional e algebra relacional

#F3006 - Apresentação

A **modelagem de dados** é o processo de estruturar e documentar como os dados são armazenados, organizados e manipulados dentro de um sistema. Ela é essencial para garantir **coerência, eficiência e integridade** no banco de dados.

Níveis da Modelagem de Dados

A modelagem ocorre em três níveis principais, cada um com um grau de detalhe diferente:

Modelo Conceitual (Visão Abstrata)

- Representação de **alto nível**, focada na estrutura dos dados sem detalhes técnicos.
- Define **entidades, atributos e relacionamentos** entre os dados.
- Usado para comunicação com stakeholders não técnicos (analistas, gestores).
- **Ferramenta comum**: Diagrama Entidade-Relacionamento (DER).
- **Exemplo**:
    - Entidade **Cliente** (com atributos: nome, CPF, telefone).
    - Entidade **Pedido** (com atributos: número, data, valor).
    - Relacionamento: **Cliente faz Pedido** (1:N).

Modelo Lógico (Estrutura Formal)

- Tradução do modelo conceitual para um formato estruturado, adequado para bancos de dados.
- Define **tabelas, colunas, chaves primárias (PK), chaves estrangeiras (FK) e relacionamentos**.
- Ainda **independente de um SGBD** específico.
- **Exemplo**:

```
sql
CopiarEditar
CREATE TABLE Cliente (
    id_cliente INT PRIMARY KEY,
    nome VARCHAR(100),
    cpf CHAR(11) UNIQUE
);

CREATE TABLE Pedido (
    id_pedido INT PRIMARY KEY,
    id_cliente INT,
    data_pedido DATE,
    valor DECIMAL(10,2),
    FOREIGN KEY (id_cliente) REFERENCES Cliente(id_cliente)
);
```

Modelo Físico (Implementação no Banco de Dados)

- **Dependente do SGBD** (MySQL, PostgreSQL, Oracle, etc.).
- Especifica **tipos de dados, índices, partições e otimizações** para performance.

- **Exemplo de otimizações**:
  - **Índices** para acelerar consultas (`CREATE INDEX idx_cliente ON Pedido(id_cliente);`).
  - **Normalização** para evitar redundância.
  - **Desnormalização** para otimizar leitura.

Princípios Fundamentais

Normalização

- Processo que organiza os dados para **evitar redundância e inconsistências**.
- Aplicação de formas normais (1ª, 2ª, 3ª FN...).
- Exemplo: Separar dados repetitivos em tabelas diferentes (ex: cliente e endereço).

Desnormalização

- Oposto da normalização: **duplica alguns dados para melhorar performance**.
- Exemplo: Em um sistema de relatórios, pode ser melhor manter algumas informações repetidas para evitar `JOIN`s complexos.

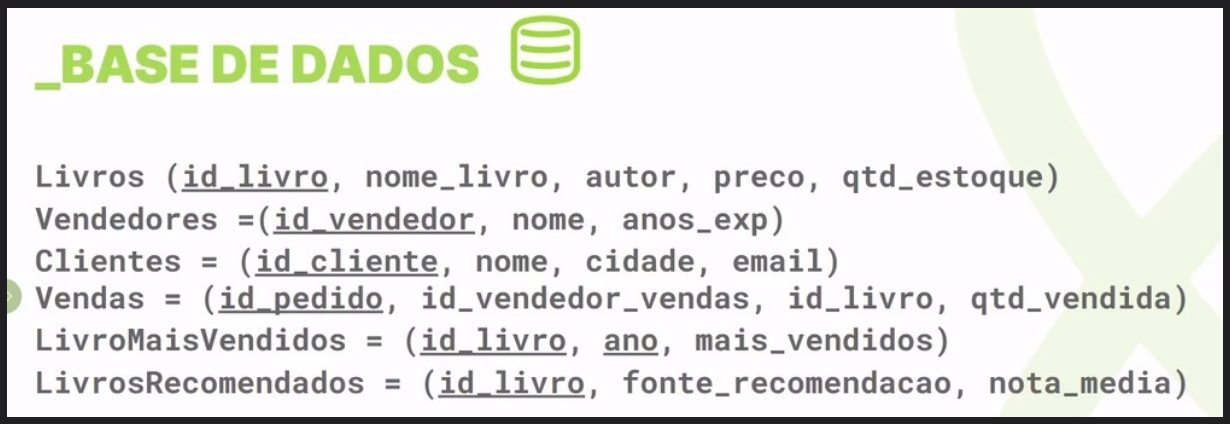
Resumo

Modelo	Objetivo	Características
Conceitual	Visão abstrata dos dados	Foca em entidades, atributos e relacionamentos
Lógico	Estrutura formal para o BD	Define tabelas, chaves e relacionamentos, mas sem detalhes do SGBD
Físico	Implementação no SGBD	Define tipos de dados, índices, otimizações

#F3007 - Clube do livro

Empresa que será utilizada como case, é um ecommerce de livro

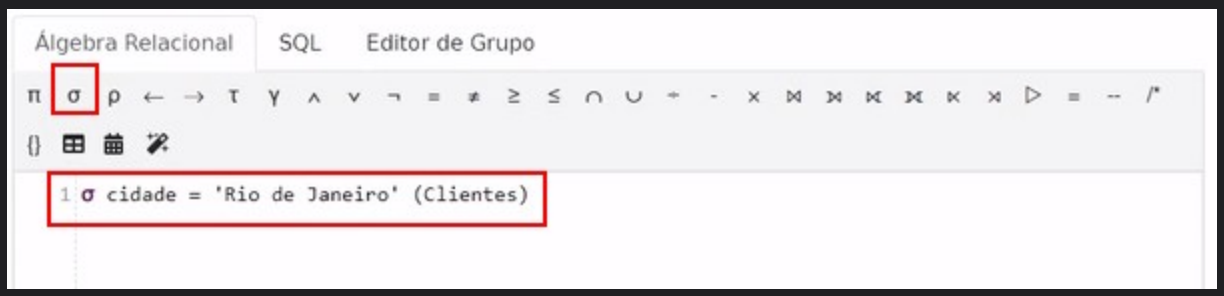
#F3008 - Ferramenta e dados



https://dbis-uibk.github.io/relax/

#F3009 – Seleção σ (sigma) – Trás linhas especificas conforme a seleção

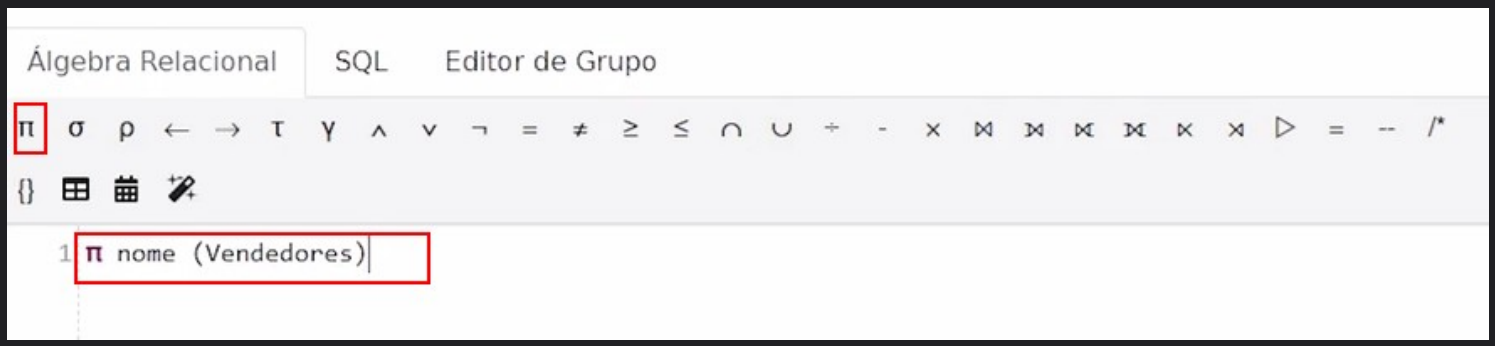
A empresa Clube do livro gostaria de saber quais clientes irão se beneficiar do frete grátis para o Rio de Janeiro?



Clientes.id_cliente	Clientes.nome	Clientes.cidade	Clientes.email
2	'Maria Santos'	'Rio de Janeiro'	'maria.santos@example.com'
12	'Carolina Menezes'	'Rio de Janeiro'	'carolina.menezes@example.com'
18	'Laura Mendes'	'Rio de Janeiro'	'laura.mendes@example.com'

#F3010 – Projeção π (pi) – Trás colunas especificas conforme projeção

Se eu quiser colunas especificas da minha relação? Como fazer isso?



Executar consulta

Download Histórico

π nome (Vendedores)

Execution time: 2 ms

Vendedores.nome
'Lucas Silva'
'Camila Santos'
'Gabriel Almeida'
'Larissa Souza'
'Rafael Lima'
'Júlia Rocha'

Álgebra Relacional SQL Editor de Grupo

π σ ρ ← → τ γ ∧ ∨ ¬ = ≠ ≥ ≤ ∩ ∪ ÷ - × ⋈ ⟕ ⟖ ⟗ ⋉ ⋊ ▷ ≡ -- /*

{} ⊞ ⊞ ✨

```
πnome,anos_exp (Vendedores)
```

Vendedores.nome	Vendedores.anos_exp
'Lucas Silva'	5
'Camila Santos'	3
'Gabriel Almeida'	7

#F3011 - Linhas e colunas específicas – Como duas opeerações podem, de formas combinadas, filtrar linhas e colunas

Álgebra Relacional SQL Editor de Grupo

```
1 π nome,email (σ cidade = 'Rio de Janeiro' (Clientes))
```

Executar seleção Download Histórico

π nome, email (σ cidade = 'Rio de Janeiro' (Clientes))

Execution time: 2 ms

Clientes.nome	Clientes.email
'Maria Santos'	'maria.santos@example.com'
'Carolina Menezes'	'carolina.menezes@example.com'
'Laura Mendes'	'laura.mendes@example.com'

#F3012 - Produto cartesiano (x)– Lista de possíveis livros para cada cliente.(é custoso tem que ter cuidado)

Álgebra Relacional SQL Editor de Grupo

```
1 π Livros.nome_livro, Clientes.nome (Livros × Clientes)
```

$\pi_{\text{Livros.nome_livro, Clientes.nome}}$ (Livros × Clientes)

Execution time: 1 ms

Livros.nome_livro	Clientes.nome
'Percy Jackson e o Ladrao de Raios'	'João Silva'
'Percy Jackson e o Ladrao de Raios'	'Maria Santos'
'Percy Jackson e o Ladrao de Raios'	'Pedro Almeida'
'Percy Jackson e o Ladrao de Raios'	'Ana Souza'
'Percy Jackson e o Ladrao de Raios'	'Carlos Lima'

#F3013 – Junção – Como saber o nome da pessoa vendedora que realizou aquela venda?

Serve para relacionar e visualizar informações que estão em tabelas diferentes.

$\pi_{\text{id_pedido, id_vendedor_vendas, id_vendedor, nome}}$ ($\sigma_{\text{id_vendedor_vendas = id_vendedor}}$ (Vendas × Vendedores))

Execution time: 2 ms

Vendas.id_pedido	Vendas.id_vendedor_vendas	Vendedores.id_vendedor	Vendedores.nome
1	3	3	'Gabriel Almeida'
2	1	1	'Lucas Silva'
3	2	2	'Camila Santos'
4	4	4	'Larissa Souza'
5	3	3	'Gabriel Almeida'
6	1	1	'Lucas Silva'
7	2	2	'Camila Santos'
8	4	4	'Larissa Souza'
9	3	3	'Gabriel Almeida'
10	1	1	'Lucas Silva'

Pesquisar com o Bing

Junção = sigma + produto

```
-- π id_pedido, id_vendedor_vendas, id_vendedor, nome (σ id_vendedor_vendas = id_vendedor (Vendas × Vendedores))

π id_pedido, id_vendedor_vendas, id_vendedor, nome (Vendas ⋈ id_vendedor_vendas = id_vendedor Vendedores)
```

#F3014 - Junção natural – Como visualizar os nomes dos livros vendidos?

É um tipo de junção que vai fazer a relação entre duas tabelas onde os atributos são iguais em ambas tabelas

π id_pedido, id_livro, nome_livro (Vendas ⋈ Livros)

Execution time: 0 ms

Vendas.id_pedido	Vendas.id_livro	Livros.nome_livro
1	7	'Gossip Girl Nao me Esqueca'
2	2	'Sangue de Lobo'
3	4	'O Simbolo Perdido'
4	8	'Cidade das Almas Perdidas'
5	6	'Pegasus e o Fogo do Olimpo'
6	6	'Pegasus e o Fogo do Olimpo'

Select DB (Clube do Livro)

Livros
- id_livro number
- nome_livro string
- autor string
- preco number
- qtd_estoque number

Vendedores
- id_vendedor number
- nome string
- anos_exp number

Clientes
- id_cliente number
- nome string
- cidade string
- email string

Vendas
- id_pedido number
- id_vendedor_vendas number
- id_livro number
- qtd_vendida number

LivroMaisVendidos
- id_livro number
- ano number
- mais_vendidos number

LivrosRecomendados
- id_livro number
- fonte_recomendacao string
- nota_media number

#F3015 - Junção esquerda e direita – Será que existe alguma pessoa vendedora que não realizou nenhuma venda?

Junção a direita

$\pi_{\text{id_vendedor_vendas, id_vendedor, nome}}$ (Vendas $\boxed{\bowtie}_{\text{id_vendedor_vendas = id_vendedor}}$ Vendedores)

Execution time: 2 ms

Vendas.id_vendedor_vendas	Vendedores.id_vendedor	Vendedores.nome
1	1	'Lucas Silva'
2	2	'Camila Santos'
3	3	'Gabriel Almeida'
4	4	'Larissa Souza'
null	5	'Rafael Lima'
null	6	'Júlia Rocha'

Trazer somente os que não venderam

$\sigma_{\text{id_vendedor_vendas = null}}$ ($\pi_{\text{id_vendedor_vendas, id_vendedor, nome}}$ (Vendas $\bowtie_{\text{id_vendedor_vendas = id_vendedor}}$ Vendedores))

Execution time: 3 ms

Vendas.id_vendedor_vendas	Vendedores.id_vendedor	Vendedores.nome
null	5	'Rafael Lima'
null	6	'Júlia Rocha'

#F3016 – União – Quais os livros mais vendidos ou mais recomendados?

LivroMaisVendidos
- id_livro number
- ano number
- mais_vendidos number

LivrosRecomendados
- id_livro number
- fonte_recomendacao string
- nota_media number

Mais vendidos:

π_{id_livro} (LivroMaisVendidos) $\cup$ π_{id_livro} (LivrosRecomendados)

Execution time: 2 ms

LivroMaisVendidos.id_livro
3
7
9
12
14
16
18
19